

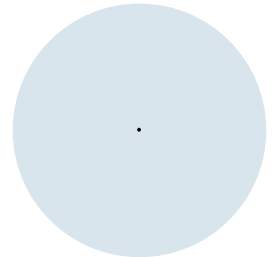
Einführung in die Programmierung (WIN+BIT)

Arrays und Referenzdatentypen

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



Arrays im Speicher



Arrays: Abbildung im Speicher

- In Java unterscheiden wir:
- **primitive Datentypen**
 - enthalten nur einen einzigen Wert ("Skalar")
 - `boolean`, `char`, `byte`, `short`, `int`, `long`, `float`, `double`
- **Referenzdatentypen** (enthalten Referenz auf einen anderen Speicherbereich mit dem Wert/den Werten)
 - Strings
 - **Arrays**
 - Klassen (später)

Unterschiedliche Speicherbereiche

Code	Stack Memory	Heap Space
<code>int n1 = 42;</code>	<code>n1: 42</code>	<code>00BC: 42</code> <code>43</code> <code>44</code>
<code>int[] array1 = {42,43,44};</code>	<code>array1: 00BC</code>	
<code>String s = "Hallo";</code>	<code>s: 00BF</code>	<code>00BF: "Hallo"</code>

Stack Memory: Speicherbereich des Hauptspeichers, in dem aktuell verfügbare Variablen und deren Werte gespeichert sind.

Heap Space: Speicher für Objekte und deren Werte (später mehr dazu), also auch Array-Werte und Strings

Wert-/Referenz-Kopien

Code

```
int n1 = 42;  
int n2 = n1;
```

```
int[] array1 = {42,43,44};  
int[] array2 = array1;
```

```
String s = "Hallo";  
String t = s;
```

Stack Memory

n1: 42
n2: 42

array1: 00BC
array2: 00BC

s: 00BF
t: 00BF

Heap Space

00BC: 42
43
44
00BF: "Hallo"

- ↪ Eintrag im Stack wird kopiert
- kann Wert eines elementaren Datentyps sein
 - oder Referenz (Verweis auf Speicherbereich im Heap Space)
- Wert-/Referenz-Kopie:** Nur Wert im **Stack Memory** wird kopiert, nicht eventuelle dahinterliegende Speicherbereiche im **Heap Space**

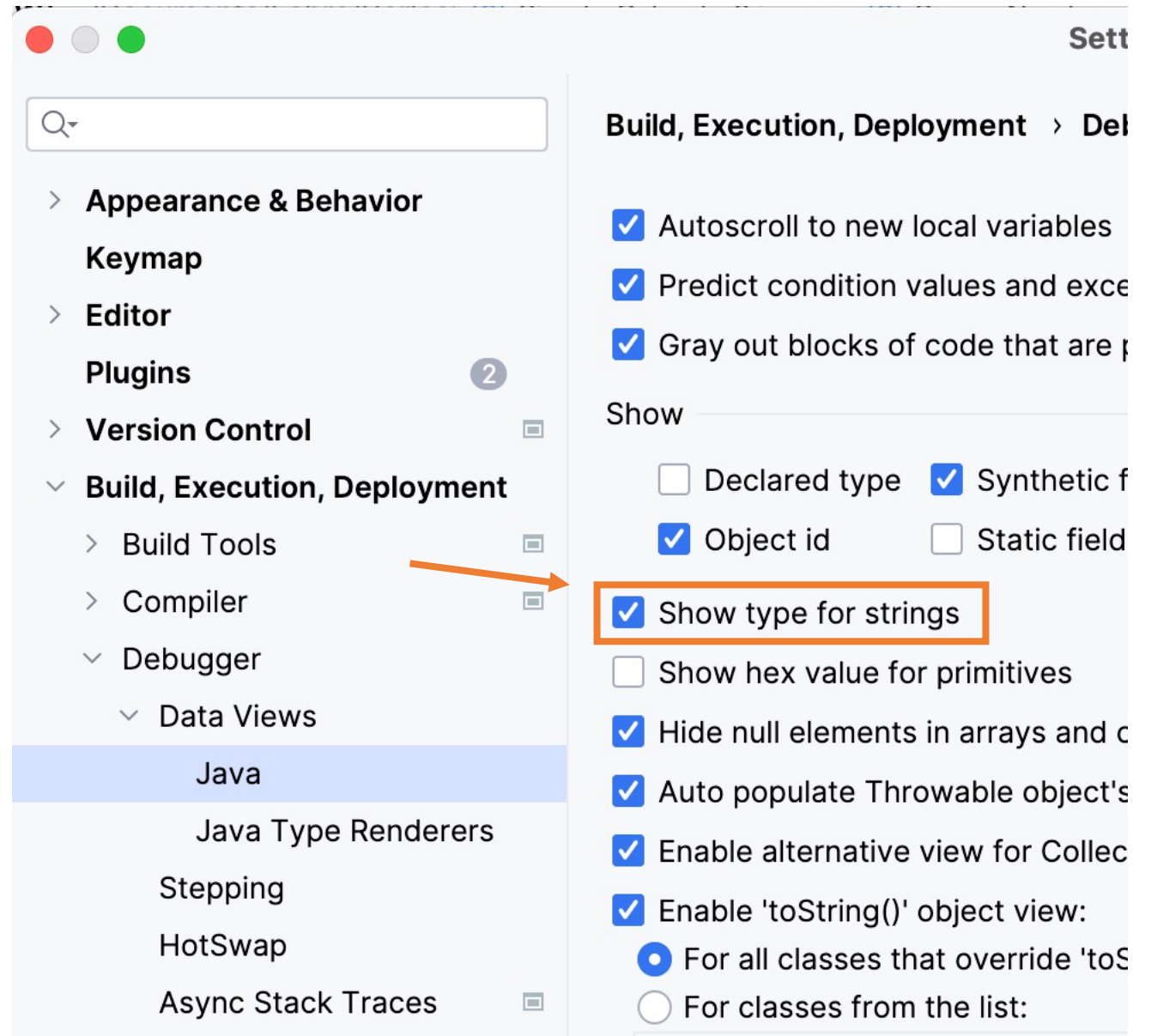
Im Debugger

- Wir sehen uns den Code unten im Debugger an. Damit man bei den Strings auch die Speicher-Referenz angezeigt bekommen, muss man unter "Settings -> Debugger -> Data Views -> Java " die Option "Show type for strings" anwählen.

```
int n1 = 42;  
int n2 = n1;
```

```
int[] array1 = {42,43,44};  
int[] array2 = array1;
```

```
String s = "Hallo";  
String t = s;
```



```

5  ▶ | public static void main(String[] args) {  args: []
6      int n1 = 42;  n1: 42
7      int n2 = n1;  n1: 42    n2: 42
8
9      int[] array1  array1: [42, 43, 44]
10         = new int[]{42,43,44};
11      int[] array2 = array1;  array1: [42, 43, 44]    array2: [42, 43, 44]
12
13      String s = "Hallo";  s: "Hallo"
14  ✓ String t = s;  s: "Hallo"    t: "Hallo"
15  } ← Code ist bis hierhin ausgeführt

```

Debug: PrimitiveVsReference ×

Debugger

Console

"main"@1... RUNNING

↑
↓
⌵

main:15, PrimitiveVsReference (session06a)

Variables

args = {String[0]@792} []

n1 = 42

n2 = 42

array1 = {int[3]@793} [42, 43, 44]

array2 = {int[3]@793} [42, 43, 44]

s = {String@794} "Hallo"

t = {String@794} "Hallo"



code09.
PrimitiveVs
Reference1

```
5 ▶ | public static void main(String[] args) {    args: []
6         int n1 = 42;    n1: 42
7         int n2 = n1;    n1: 42    n2: 42
8
9         int[] array1    array1: [42, 43, 44]
10            = new int[]{42,43,44};
11         int[] array2 = array1;    array1: [42, 43, 44]    array2: [42, 43, 44]
12
13         String s = "Hallo";    s: "Hallo"
14         String t = s;    s: "Hallo"    t: "Hallo"
15     }
```

Debug:  PrimitiveVsReference ✕

Debugger Console  **Inhalt Stack** **Inhalt Heap**

Variables

```
+ p args = {String[0]@792} []  
01 n1 = 42  
01 n2 = 42  
> 1/3 array1 = {int[3]@793} [42, 43, 44]  
> 1/3 array2 = {int[3]@793} [42, 43, 44]  
> s = {String@794} "Hallo"  
> t = {String@794} "Hallo"
```

Wert einer primitiven Variable

Referenz auf Heap-Speicher-Adressen (793 u. 794)



```
code09.  
PrimitiveVs  
Reference1
```

Zugriff auf primitive Variable

Code

```
int n1 = 42;  
int n2 = n1;
```

```
int[] array1 = {42,43,44};  
int[] array2 = array1;
```

```
String s = "Hallo";  
String t = s;
```

Stack Memory

```
n1: 42  
n2: 42
```

```
array1: 00BC  
array2: 00BC
```

```
s: 00BF  
t: 00BF
```

Heap Space

```
00BC: 42  
      43  
      44
```

```
00BF: "Hallo"
```

Zugriff auf n1:

- in Stack unter n1 nachsehen: Dort steht der Wert 42
- Also `n1 == 42`.

Zugriff auf Array

Zugriff auf Array

Code	Stack Memory	Heap Space	Array-Index
<code>int[] array1 = {42,43,44};</code>	<code>array1: 00BC</code>	<code>00BC: 42</code>	0
<code>int[] array2 = array1;</code>	<code>array2: 00BC</code>	<code>43</code>	1
		<code>44</code>	2

Zugriff auf `array1[1]`

- in Stack unter `array1` nachsehen: Dort steht Referenz auf Speicherbereich `00BC`
- in Speicherbereich `00BC` an Index `[1]` nachsehen: Dort steht `43`.
- also `array1[1] == 43`

Zugriff auf `array2[1]` genauso.

- also `array2[1] == 43 == array1[1]`

Verändern des Array-Inhalts

Verändern des Array-Inhalts

Code	Stack Memory	Heap Space	Array-Index
int[] array1 = {42,43,44};	array1: 00BC	00BC: 42	0
int[] array2 = array1;	array2: 00BC	0	1
array2[1] = 0;		44	2

Verändern von array2[1]:

- in Stack unter array2 nachsehen: Dort steht Referenz auf Speicherbereich 00BC
- in Speicherbereich 00BC auf Position [1] zugreifen und Wert ändern.

Resultat nach Verändern

Code	Stack Memory	Heap Space	Array-Index
int[] array1 = {42,43,44};	array1: 00BC	00BC: 42 0 44	0
int[] array2 = array1;	array2: 00BC		1
array2[1] = 0;			2

Zugriff auf array1[1] oder array2[1]:

- in Stack unter array1/array2 nachsehen: Dort steht Referenz auf Speicherbereich 00BC
- in Speicherbereich 00BC an Position [1] nachsehen: Dort steht 43.
- also array1[1] == array2[1] = 0
- **Also hat sich auch der Wert für array1[1] geändert!!!**



code09.
PrimitiveVs
Reference2

Primitiver Typ vs. Referenztyp

Vorsicht bei Vergleichen und Zuweisungen!

Primitiver Typ

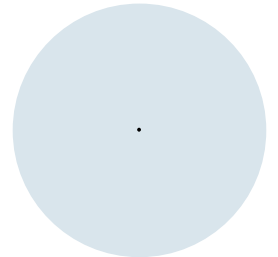
- **Primitive Typ**
- Variable wird mit Wert initialisiert
`double pi = 3.141592653589`
- Variable beinhalten Wert (z.B. eine Zahl),
dieser Wert...
 - wird bei Zuweisungen kopiert,
 - wird bei Vergleichen verglichen.

haben wir
gezeigt

Referenz-Typ

- **Referenz-Typ**
- Heap-Speicherbereich für Variable wird mit
new initialisiert
`int[] numbers;`
`numbers = new int[3];`
- Variable beinhalten Speicheradresse des
Heap-Speichers). Nicht der Inhalt des Heap-
Speichers, sondern diese Adresse...
 - wird bei Zuweisungen kopiert,
 - wird bei Vergleichen verglichen.

Methodenaufrufe, Stack versus Heap



Wirkung von Methodenaufrufen

Methodenaufrufe wie

```
doSomething(argument);
```

- Auf welche Speicherbereiche haben sie Einfluss?
- Wie können sie argument verändern, so dass die Änderungen auch außerhalb der Methode sichtbar sind?

Hinweis zu den Beispielen

- Es gibt für die folgenden Beispiele auch eine Variante `<Beispiel>MorePrintln`
 - `ParameterPassingInt -> ParameterPassingIntMorePrintln`
 - ...
- Diese enthalten mehr `System.out.println`-Ausgaben (ggf. leichter zu verstehen für die Nachbereitung)

Aufruf mit primitivem Parameter

```
void doSomething(int intParam) {  
    intParam = 11;  
}  
  
void main() {  
    int intMain = 42;  
    doSomething(intMain);  
    System.out.println(intMain);  
}
```



code09.

Parameter
PassingInt

Aufruf mit primitivem Parameter

Code

```
int intMain = 42;
```

Stack Memory

```
intMain: 42
```

Heap Space

- Wert im Stack



code09.

Parameter
PassingInt

Aufruf mit primitivem Parameter

Code

```
int intMain = 42;  
doSomething(intMain) {
```

Stack Memory

```
intMain: 42  
intParam: 42
```



Heap Space

- Für den Aufruf wird der Inhalt von intMain im Stack nach intParam kopiert.



code09.

Parameter
PassingInt

Aufruf mit primitivem Parameter

Code

```
int intMain = 42;  
doSomething(intMain) {  
    intParam = 11;  
}
```

Stack Memory

```
intMain: 42  
intParam: 11
```

Heap Space

- innerhalb doSomething wird intParam im Stack geändert



Code09.

Parameter
PassingInt

Aufruf mit primitivem Parameter

Code

```
int intMain = 42;  
doSomething(intMain);  
System.out  
    .println(intMain);
```

Stack Memory

intMain: 42

Heap Space

- zurück in der Main-Methode liefert intMain immer noch den Wert 42.
- der Stack-Teil mit intParam hat seine Gültigkeit verloren (Werte werden wieder aus dem Stack entfernt)



code09.

Parameter
PassingInt

Inhaltsänderung Array

```
public static void doSomething(int[] arrayParam) {  
    arrayParam[1] = 11;  
}  
  
public static void main(String[] args) {  
    int[] arrayMain = new int[]{42, 43};  
    doSomething(arrayMain);  
    System.out.println(Arrays.toString(arrayMain));  
}
```

Hier wird auf den über
arrayParam referenzierten
Speicherbereich zugegriffen.



code09.

Parameter
Passing
IntArray

Inhaltsänderung Array

Code

```
int[] arrayMain =  
    new int[] {42,43};  
doSomething(arrayMain) {  
    arrayParam[1] = 11;  
}
```

Stack Memory

arrayMain: 00AB

arrayParam: 00AB



Heap Space

00AB:	42
	43 11

→ Referenz auf Array

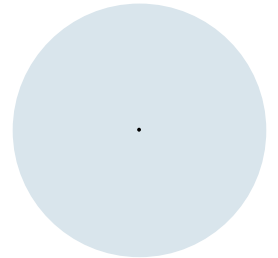
- Inhaltsänderung des referenzierten Objekts klappt auch, wenn das referenzierte Objekt ein Array ist



code09.

Parameter
Passing
IntArray

Arraykopie



Resultat nach Verändern

Code	Stack Memory	Heap Space	Array-Index
<code>int[] array1 = {42,43,44};</code>	<code>array1: 00BC</code>	<code>00BC: 42</code>	0
<code>int[] array2 = array1;</code>	<code>array2: 00BC</code>	0	1
<code>array2[1] = 0;</code>		44	2

array1 und array2 zeigen nach der Zuweisung `array2 = array1` auf denselben Speicherbereich im Heap Space, da hier nur die **Referenz** (=Speicheradresse des Heaps) kopiert wird.

→ Man kann array1 nicht unabhängig von array2 ändern. Änderungen, z.B. von `array2[1]` wirken sich auf beide Variablen aus.



code09.
PrimitiveVs
Reference2

Primitiver Typ vs. Referenztyp

Vorsicht bei Vergleichen und Zuweisungen!

Primitiver Typ

- **Primitive Typ**
- Variable wird mit Wert initialisiert
`double pi = 3.141592653589`
- Variable beinhalten Wert (z.B. eine Zahl),
dieser Wert...
 - wird bei Zuweisungen kopiert,
 - wird bei Vergleichen verglichen.

haben wir
gezeigt

Wie können wir den **Inhalt des
Arrays** kopieren?

Referenz-Typ

- **Referenz-Typ**
- Heap-Speicherbereich für Variable wird mit
new initialisiert
`int[] numbers;`
`numbers = new int[3];`
- Variable beinhalten Speicheradresse des
Heap-Speichers). Nicht der Inhalt des Heap-
Speichers, sondern diese Adresse...
 - wird bei Zuweisungen kopiert,
 - wird bei Vergleichen verglichen.

Arraykopie

Situation:

- Wir erhalten ein int-Array mit Werten.
- Wir möchten dieses Array kopieren.
- Anschließend möchten wir mit der Kopie arbeiten, dort Werte verändern (im Beispiel die Werte sortieren), ohne dass das Original verändert wird.

Arraykopie

```
int[] originalValues = { 4, 2, 1, 3, 6, 5 };
int[] sortedValues = ???; // create copy

System.out.println("Vor Sortierung:");
System.out.println("originalValues: " + Arrays.toString(originalValues));
System.out.println("sortedValues:   " + Arrays.toString(sortedValues));

Arrays.sort(sortedValues); // sortedValues wird verändert!

System.out.println("Nach Sortierung:");
System.out.println("originalValues: " + Arrays.toString(originalValues));
System.out.println("sortedValues:   " + Arrays.toString(sortedValues));
```

Zielzustand nach Kopie

Code

```
int[] originalValues
= {4,2,1,3,6,5};
int[] sortedValues
= ??? //Kopie
```

Stack Memory

originalValues: 00BC
sortedValues: 00C0

Heap Space

00BC: 4	0
2	1
1	2
3	3
6	4
5	5
00C0: 4	0
2	1
1	2
3	3
6	4
5	5

Array-Index



Arraykopie (1. Versuch)

```
int[] originalValues = { 4, 2, 1, 3, 6, 5 };  
// Kopie der Array-Referenz  
int[] sortedValues = originalValues;  
  
System.out.println("Vor Sortierung:");  
System.out.println("originalValues: " + Arrays.toString(originalValues));  
System.out.println("sortedValues:   " + Arrays.toString(sortedValues));  
  
Arrays.sort(sortedValues);  
  
System.out.println("Nach Sortierung:");  
System.out.println("originalValues: " + Arrays.toString(originalValues));  
System.out.println("sortedValues:   " + Arrays.toString(sortedValues));
```

Istzustand nach einfacher Zuweisung

Code

```
int[] originalValues  
    = {4,2,1,3,6,5};  
int[] sortedValues  
    = originalValues
```

Stack Memory

originalValues: 00BC
sortedValues: 00BC

Heap Space

4	0
2	1
1	2
3	3
6	4
5	5

Array-Index

sortedValues und originalValues zeigen nach der **Zuweisung** auf denselben Speicherbereich im Heap Space, da hier nur die **Referenz** (=Speicheradresse des Heaps) kopiert wird.

→ Man kann array1 nicht unabhängig von array2 ändern. Änderungen, z.B. von `sortedValues[1]` wirken sich auf beide Variablen aus.



code09.
ArrayCopy
Values1

Arraykopie (2. Versuch)

```
int[] originalValues = { 4, 2, 1, 3, 6, 5 };  
// neuen Heap-Speicherbereich initialisieren  
int[] sortedValues = new int[originalValues.length];  
// Werte kopieren  
for (int i = 0; i < originalValues.length; i++) {  
    sortedValues[i] = originalValues[i];  
}  
  
System.out.println("Vor Sortierung:");  
System.out.println("originalValues: " + Arrays.toString(originalValues));  
System.out.println("sortedValues:    " + Arrays.toString(sortedValues));  
  
Arrays.sort(sortedValues);  
  
System.out.println("Nach Sortierung:");  
System.out.println("originalValues: " + Arrays.toString(originalValues));  
System.out.println("sortedValues:    " + Arrays.toString(sortedValues));
```

Istzustand nach **erfolgreicher** Kopie

Code

```
int[] originalValues
= {4,2,1,3,6,5};
int[] sortedValues
= new int[6]
// ... (Kopie der Einträge)
```

Das Schlüsselwort **new** erzeugt ein Array an einem neuen (freien) Speicherplatz

Stack Memory

originalValues: 00BC → 00BC:
sortedValues: 00C0

Heap Space

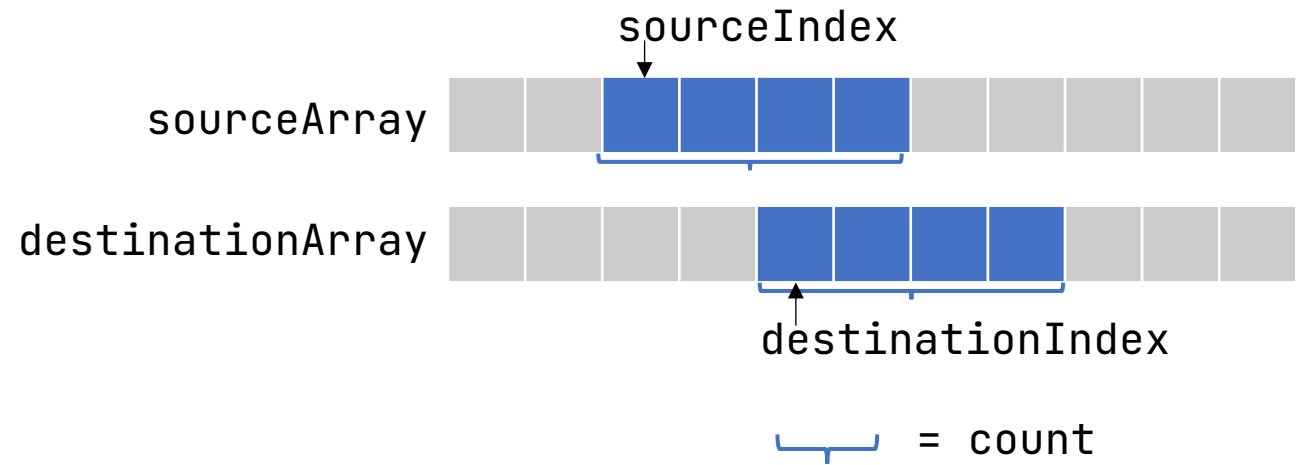
4	0
2	1
1	2
3	3
6	4
5	5

4	0
2	1
1	2
3	3
6	4
5	5

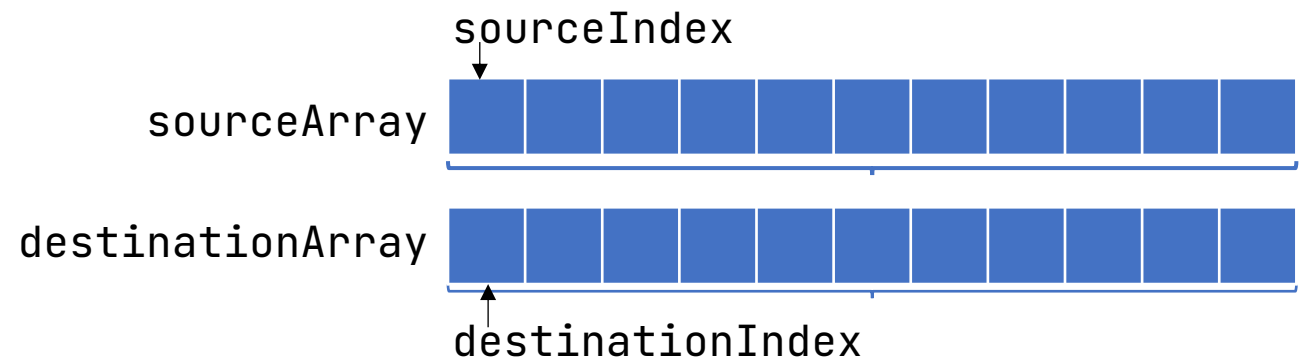
Array-Index

Alternative zur Schleife: `System.arraycopy(...)`

- `System.arraycopy(
 <sourceArray>,
 <sourceIndex>,
 <destinationArray>,
 <destinationIndex>,
 <count>)`



System.arraycopy(...)



 = count

```
int[] originalValues = { 4, 2, 1, 3, 6, 5 };
int[] sortedValues = new int[originalValues.length];
System.arraycopy(
    originalValues,          // sourceArray
    0,                       // sourceIndex
    sortedValues,            // destinationArray
    0,                       // destinationIndex
    originalValues.length); // count
```



code09.
ArrayCopy
Values2

Arrays.copyOf(...)

- bequemere Varianten von `System.arraycopy()`
- erzeugen das neue Array direkt mit
- decken zwei übliche Fälle ab
 - `Arrays.copyOfRange(original, from, to)`
 - erzeugt neues Array mit Kopie der Werte von Position `from` (inklusive) bis `to` (exklusiv)
 - `Arrays.copyOf(original, newLength)`
 - erzeugt neues Array mit Kopie der Werte
 - fügt ggf. Standard-Werte (z.B. Zahl 0) hinten an oder schneidet das Array hinten ab, je nach Wahl von `newLength`
- Kann man beide auch zum Kopieren des ganzen Arrays verwenden:

```
int[] sortedValues = Arrays.copyOfRange(originalValues, 0, originalValues.length);
```

– oder

```
int[] sortedValues = Arrays.copyOf(originalValues, originalValues.length);
```

Alternative mit clone()

- `destinationArray =
 sourceArray.clone();`

```
int[] originalValues = { 4, 2, 1, 3, 6, 5 };  
int[] sortedValues = originalValues.clone();
```



code09.
ArrayCopy
Values4

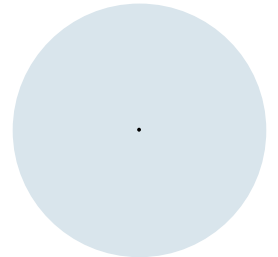
Vergleich der Kopiermethoden

- `sourceArray.clone()`
 - Flache Kopie aller Werte im Array
- `System.arraycopy()` oder die bequemerer Varianten `Arrays.copyOfRange()` und `Arrays.copyOf()`
 - Flache Kopie aller/ausgewählter Werte im Array
- Manuell mit Schleife
 - Flache/Tiefe Kopie aller/ausgewählter Werte im Array

Flache Kopien

- Flache Kopie kopiert Werte der Elemente
 - Ok für **primitive** Typen
 - Bei **Referenztypen** nur **Kopie der Referenzen**, nicht der referenzierten Werte/Objekte!
- Problematisch bei
 - Arrays von Objekten (noch für den Hinterkopf)
 - **Mehrdimensionalen** Arrays (s. später)

Array-Vergleich



Primitiver Typ vs. Referenztyp

Vorsicht bei Vergleichen und Zuweisungen!

Primitiver Typ

- **Primitive Typ**
- Variable wird mit Wert initialisiert
`double pi = 3.141592653589`
- Variable beinhalten Wert (z.B. eine Zahl),
dieser Wert...
 - wird bei Zuweisungen kopiert,
 - wird bei Vergleichen verglichen.

betrachten
wir jetzt

Referenz-Typ

- **Referenz-Typ**
- Heap-Speicherbereich für Variable wird mit
new initialisiert
`int[] numbers;`
`numbers = new int[3];`
- Variable beinhalten Speicheradresse des
Heap-Speichers). Nicht der Inhalt des Heap-
Speichers, sondern diese Adresse...
 - wird bei Zuweisungen kopiert,
 - wird bei Vergleichen verglichen.

Vergleich von Referenzen

- Was ist die Ausgabe und warum?

```
int[] valuesA = {1, 2};  
int[] valuesB = {1, 2};  
  
if (valuesA == valuesB) {  
    System.out.println("Gleichheit");  
} else {  
    System.out.println("Ungleichheit");  
}
```



code09.
ArrayRefer-
enceEquals

```

5 ▶ public static void main(String[] args) { args: []
6     int[] valuesA = {1, 2}; valuesA: [1, 2]
7     int[] valuesB = {1, 2}; valuesB: [1, 2]
8
9
10
11
12     if (valuesA == valuesB) { valuesA: [1, 2] valuesB: [1, 2]
13         System.out.println("Gleichheit");
14     } else {
15         System.out.println("Ungleichheit");
16     }
17 }

```

Debug: GleichOderUngleich x

Inhalt Stack Inhalt Heap

Debugger Console

Frames

Variables

✓ "main"...RUNNING

main:17, GleichOderUngleich (session06b)

args = {String[0]@791} []

valuesA = {int[2]@792} [1, 2]

valuesB = {int[2]@793} [1, 2]

== vergleicht den Inhalt des Stacks → unterschiedliche Speicheradressen!

valuesA und valuesB im Speicher

Code

```
int[] valuesA = {1, 2};  
int[] valuesB = {1, 2};
```

Stack Memory

valuesA: 00BC

valuesB: 00BF

Heap Space

00BC: 1
2

00BF: 1
2

Stack Memory: Speicherbereich des Hauptspeichers, in dem aktuell verfügbare Variablen und deren Werte gespeichert sind.

Heap Space: Speicher für Objekte und deren Werte, also auch Array-Werte

Die Darstellung ist stark vereinfacht und dient zur Veranschaulichung des Speichermodells von Java. Wir werden sie an vielen Stellen verwenden.

valuesA und valuesB im Speicher

Code

```
int[] valuesA = {1, 2};  
int[] valuesB = {1, 2};
```

Stack Memory

valuesA: 00BC

valuesB: 00BF

Heap Space

00BC: 1
2

00BF: 1
2

- Zugriff auf valuesA liefert den Wert aus dem Stack, also die Speicheradresse 00BC. Zugriff auf valuesB liefert den Wert aus dem Stack, also die Speicheradresse 00BF.
- valuesA == valuesB vergleicht also 00BC mit 00BF.
- Unterschiedliche Werte, also Ungleichheit!

Vergleich von Referenzen

- Hier erhalten also wir "Ungleichheit", da nur die Referenzen verglichen werden

```
int[] valuesA = {1, 2};  
int[] valuesB = {1, 2};  
  
if (valuesA == valuesB) {  
    System.out.println("Gleichheit");  
} else {  
    System.out.println("Ungleichheit");  
}
```

- Was aber, wenn wir wirklich den Inhalt vergleichen wollen?



code09.
ArrayRefer-
enceEquals

Vergleich des Inhalts

Manuell vergleichen

- zuerst, ob Länge gleich
- dann jeden Platz
- am Ende Ergebnis in Variable `arraysAreEqual`

```
//          id 0  1
int[] valuesA = {1, 2};
int[] valuesB = {1, 2};

// Array-Inhalt vergleichen
boolean arraysAreEqual = true;
if (valuesA.length != valuesB.length) {
    arraysAreEqual = false;
} else {
    // jeden Platz vergleichen
    for (int i = 0; i < valuesA.length; i++) {
        if (valuesA[i] != valuesB[i]) {
            arraysAreEqual = false;
            break;
        }
    }
}
System.out.println(arraysAreEqual ?
    "Arrays sind gleich" : "Arrays sind ungleich");
```

Vergleich des Inhalts

- Einfacher: Wir verwenden die vorgefertigte Methode `Arrays.equals(array1, array2)`

```
int[] valuesA = {1, 2};  
int[] valuesB = {1, 2};  
  
// Array-Inhalt vergleichen  
boolean arraysAreEqual = Arrays.equals(valuesA, valuesB);  
System.out.println(arraysAreEqual ?  
    "Arrays sind gleich" : "Arrays sind ungleich");
```



code09.
ArrayContent
Equals